

Rapport – Livrable 3

Équipe

- Wendkuni Reine Rosine Guinguere
 - Meriem Ghaoui
 - Samah Tanisha Islam
-

Tâche 1 - Fonctionnalité additionnelle

1. Objectif

Pour la troisième phase du projet, nous avons ajouté une fonctionnalité de recommandation de lieux. L'objectif était de permettre à l'utilisateur de trouver plus facilement un endroit qui correspond à l'ambiance qu'il recherche, plutôt que de devoir consulter les différents lieux un par un.

Cette fonctionnalité utilise les données d'ambiance qui étaient déjà disponibles dans l'application afin de proposer un lieu selon le besoin sélectionné par l'utilisateur.

2. Fonctionnement de la recommandation

La page de recommandation permet à l'utilisateur de sélectionner le type d'environnement qu'il recherche. Il peut par exemple chercher un endroit adapté à l'étude et à la concentration, un environnement pour travailler en groupe ou un endroit avec une ambiance plus animée.

Ces différents choix sont associés aux classifications d'ambiance déjà utilisées dans l'application, comme calm, normal, busy et noisy.

À partir du choix de l'utilisateur, l'application récupère les informations d'ambiance disponibles pour les différents lieux et vérifie lesquels correspondent au besoin sélectionné.

Lorsqu'il y a plusieurs lieux possibles, la date des dernières données disponibles est également prise en compte. Le but est de privilégier un lieu dont les informations d'ambiance sont plus récentes.

3. Affichage du résultat

Une fois qu'un lieu correspondant a été trouvé, l'application affiche le lieu recommandé avec une justification permettant à l'utilisateur de comprendre pourquoi cette recommandation lui est proposée.

L'utilisateur peut ensuite accéder directement au portrait du lieu recommandé afin de consulter plus d'informations sur celui-ci.

4. Intérêt de la fonctionnalité

Nous avons choisi cette fonctionnalité parce qu'elle permet de mieux utiliser les données déjà présentes dans l'application. Avant, l'utilisateur pouvait principalement consulter les informations d'ambiance des différents lieux. Avec la recommandation, ces mêmes données peuvent maintenant l'aider à choisir un lieu.

Par exemple, une personne qui cherche un endroit calme pour étudier peut sélectionner ce besoin et obtenir directement une suggestion, au lieu d'ouvrir plusieurs portraits de lieux pour comparer leur ambiance.

La fonctionnalité reste donc liée au but principal de l'application, mais ajoute une utilisation plus personnalisée des données disponibles.

Tâche 2 — Maintenabilité et réutilisabilité

2.1 - Backend

Objectif

L'objectif de cette tâche était d'améliorer la structure du backend en séparant la logique métier des contrôleurs HTTP. Avant la refactorisation, plusieurs contrôleurs contenaient à la fois :

- la lecture des paramètres de requête ;
- la validation des données ;
- les appels à MongoDB ;
- les calculs métier ;
- la préparation des réponses HTTP.

Cette organisation rendait le code plus difficile à tester et à maintenir. La logique métier a donc été déplacée dans des services réutilisables.

Séparation des responsabilités

La structure retenue est la suivante :

- **Routes** : définissent les URLs et les middlewares utilisés.
- **Contrôleurs** : lisent la requête, appellent un service et renvoient la réponse HTTP.
- **Services** : contiennent la logique métier, les validations et les appels aux modèles.
- **Modèles** : définissent les schémas Mongoose et la structure des données MongoDB.

Après la refactorisation, les contrôleurs ne contiennent plus les calculs métier. Ils se limitent principalement à :

- lecture de `req.params`, `req.query` ou `req.body`
- appel du service

- réponse HTTP
- gestion des erreurs

Cette séparation permet de réutiliser les services dans d'autres contrôleurs ou dans d'autres parties de l'application.

Refactorisation du service d'ambiance

Le service d'ambiance a été choisi en premier, car `ambianceController.js` contenait une quantité importante de logique métier :

- normalisation des noms de lieux ;
- classification du niveau d'ambiance ;
- calcul de la moyenne sonore ;
- sélection des mesures d'un lieu ;
- sélection des observations ;
- calcul des heures calmes ;
- calcul de la fraîcheur des données ;
- génération des prévisions.

Cette logique a été déplacée dans : `src/services/ambianceService.js`

Les fonctions principales du service sont :

- `normalize`
- `classifyAmbianceRelative`
- `classifyProjectedLevel`
- `getConfidence`
- `calculateAmbianceSummary`
- `calculateQuietHours`
- `calculateAmbianceForecast`

Fonction normalize

La fonction `normalize` transforme un nom de lieu dans un format standard :

"Salle 123" → "salle-123"

Elle convertit le texte en minuscules, supprime les espaces inutiles et remplace les espaces et les underscores par des tirets. Cette fonction est une logique utilitaire réutilisable. Elle ne doit donc pas rester dans le contrôleur.

Fonctions de classification

Les fonctions suivantes ont également été retirées du contrôleur :

- `classifyAmbianceRelative`
- `classifyProjectedLevel`

Elles contiennent des règles métier permettant de déterminer le niveau d'ambiance à partir des valeurs sonores observées ou projetées. Comme ces fonctions ne dépendent pas directement de req ou de res, elles peuvent être testées séparément sans démarrer le serveur.

Calcul du résumé d'ambiance

Le calcul du résumé d'ambiance a été déplacé dans le service. Le service est responsable de :

- normaliser le lieu demandé ;
- récupérer les mesures ;
- filtrer les mesures associées au lieu ;
- récupérer les observations ;
- calculer la moyenne sonore ;
- déterminer le niveau d'ambiance ;
- identifier la mesure et l'observation les plus récentes ;
- calculer la fraîcheur des données.

Le contrôleur ne réalise plus directement ces opérations.

Calcul des heures calmes

La logique de `getQuietHours` a également été déplacée dans le service. Le service :

- récupère les mesures du lieu ;
- regroupe les mesures par heure ;
- calcule les moyennes horaires ;
- identifie les créneaux les moins bruyants.

Le contrôleur appelle seulement le service et renvoie le résultat.

Prévision d'ambiance

La logique de `getAmbianceForecast` a été isolée dans le service. Le service s'occupe de :

- limiter le nombre d'heures demandées ;
- récupérer les mesures historiques ;
- regrouper les données par heure ;

- classier les niveaux projetés ;
- calculer la confiance ;
- créer les résultats de prévision.

Cela permet de tester la projection indépendamment d'Express.

Refactorisation de userService

La logique d'authentification a été déplacée de `UserController.js` vers : `src/services/userService.js`

Le service contient notamment :

- `generateToken`
- `formatUser`
- `registerUserService`
- `loginUserService`

Inscription

La fonction `registerUserService` prend en charge :

- la validation des champs ;
- la normalisation de l'adresse courriel ;
- la recherche d'un utilisateur existant ;
- le hachage du mot de passe ;
- la création de l'utilisateur ;
- la génération du token JWT.

Le contrôleur ne fait maintenant qu'envoyer `req.body` au service et retourner la réponse.

Connexion

La fonction `loginUserService` prend en charge :

- la validation des données ;
- la recherche de l'utilisateur ;
- la comparaison du mot de passe avec `bcrypt`;
- la génération du token JWT.

La génération du token a été isolée dans `generateToken`, ce qui permet de la tester indépendamment. La fonction `formatUser` prépare également une version publique de l'utilisateur sans exposer son mot de passe.

Refactorisation de measurementService

La logique des mesures a été déplacée dans : src/services/measurementService.js

Le service contient :

- createMeasurementService
- getMeasurementsService

createMeasurementService s'occupe de :

- vérifier la présence de soundLevel et amplitude ;
- vérifier la présence de deviceId ;
- créer la mesure ;
- utiliser un timestamp par défaut si nécessaire.

getMeasurementsService s'occupe de :

- récupérer les mesures ;
- ajouter les informations publiques du device avec populate;
- trier les mesures par createdAt décroissant.

Le contrôleur conserve uniquement la lecture de la requête, l'appel du service et la réponse HTTP.

Refactorisation de observationService

La logique des observations a été déplacée dans : src/services/observationService.js

Le service contient :

- createObservationService
- getObservationsService

Les données de l'observation et les informations d'authentification sont transmises séparément :

```
createObservationService(  
  observationData,  
  authData  
);
```

Les données de l'observation comprennent :

- location
- vibe
- sourceProximity
- notes

- timestamp

locationId

Les informations d'authentification comprennent :

- authType
- deviceId
- authorId

Si la requête vient d'un device, l'observation est associée à deviceId. Si elle vient d'un utilisateur, elle est associée à author. Si aucune association n'est applicable, les deux champs restent à null.

Cette organisation évite que le service dépende directement de req, res, req.device ou req.user.

Refactorisation de deviceService

La logique des devices a été déplacée dans :src/services/deviceService.js

Le service contient notamment :

- createDeviceService
- getDevicesService
- formatDeviceWithApiKey

createDeviceService :

- vérifie name et location ;
- génère une clé API ;
- crée le device ;
- retourne les informations nécessaires au client.

formatDeviceWithApiKey prépare la réponse après la création du device.

getDevicesService utilise :

```
Device.find().select("-apiKey");
```

La clé API n'est donc pas exposée lors de la récupération générale des devices.

Refactorisation de locationService

La logique des lieux a été déplacée dans :src/services/locationService.js

Le service contient :

- getLocationsService
- createLocationService

getLocationsService :

- récupère les lieux ;
- sélectionne les champs publics ;
- trie les lieux par nom.

createLocationService :

- vérifie les champs obligatoires ;
- normalise idlocation en minuscules ;
- vérifie si le lieu existe déjà ;
- crée le lieu.

La vérification de latitude et longitude utilise explicitement :

latitude === **undefined**

longitude === **undefined**

Cela permet d'accepter correctement la valeur 0, qui est une coordonnée valide.

Résultat de la refactorisation

La refactorisation permet :

- de réduire la taille des contrôleurs ;
- d'éviter la duplication de logique ;
- de rendre les fonctions métier réutilisables ;
- de tester les calculs sans serveur ;
- de clarifier les responsabilités entre les couches ;
- de faciliter l'ajout de nouvelles fonctionnalités ;
- de limiter les risques de régression sur les endpoints existants.

Les endpoints existants continuent de fonctionner, car les routes conservent leurs contrôleurs et les contrôleurs conservent les mêmes réponses HTTP. La différence principale est que la logique métier est maintenant exécutée par des services dédiés.

2.2 – Frontend

Maintenabilité et réutilisabilité du frontend

Dans le frontend, les fonctionnalités sur lesquelles nous avons principalement travaillé concernent l'authentification, l'espace compte et le système de recommandation. Lors des modifications apportées pendant la troisième phase, nous avons essayé de conserver le

fonctionnement des fonctionnalités déjà présentes tout en intégrant la nouvelle page de recommandation.

Authentification et espace compte

L'authentification est utilisée dans plusieurs parties de l'espace compte afin de permettre à l'utilisateur d'accéder à ses informations personnelles. Les pages liées aux favoris, aux lieux et aux contributions sont accessibles à partir du même espace compte et dépendent de l'utilisateur connecté.

Pendant cette phase, ces différents parcours ont été vérifiés afin de s'assurer que les changements apportés au projet n'avaient pas affecté leur fonctionnement. Les redirections vers les pages qui nécessitent une connexion ont également été vérifiées.

Cette organisation permet de garder les fonctionnalités associées au compte utilisateur regroupées dans le même espace plutôt que de les répartir dans plusieurs parties de l'application.

Tâche 3 — Tests unitaires des services

Objectif

L'objectif de cette tâche était de configurer un outil de tests unitaires, de couvrir les services backend et de vérifier que les principales règles métier fonctionnent correctement sans dépendre du serveur Express ni d'une vraie base de données MongoDB. Les tests ont été réalisés avec **Vitest**, un outil adapté au projet et compatible avec la structure utilisée. Les fonctions asynchrones sont testées avec `async`, `await` et `mockResolvedValue`, ce qui permet à Vitest d'attendre correctement la résolution des promesses.`main.vitest`

Configuration de Vitest

Vitest a été installé comme dépendance de développement :

```
npm install --save-dev vitest
```

La commande suivante a été ajoutée dans `package.json` :

```
{ "scripts": {  
  "test": "vitest run" }}
```

La commande utilisée pour exécuter les tests est donc :

```
npm test
```

L'option `run` permet d'exécuter les tests une seule fois et de quitter automatiquement le processus, ce qui est adapté à une vérification avant une fusion ou une remise.

Organisation des tests

Chaque fichier de test regroupe les tests d'un service avec `describe` :

```
describe("ambianceService", () => {  
  // tests du service });
```

Les fonctions principales utilisées sont :

- describe : regroupe des tests liés à un même service ou à une même fonction ;
- it : définit un test individuel ;
- expect : vérifie qu'un résultat correspond à la valeur attendue ;
- vi.fn : crée une fonction simulée ;
- vi.spyOn : observe ou remplace temporairement une méthode existante ;
- afterEach : restaure les mocks après chaque test.

Les appels aux modèles Mongoose sont simulés avec vi.spyOn. Les tests vérifient donc la logique des services sans créer de données dans MongoDB. Les mocks permettent notamment de simuler Model.create, Model.find, Model.findOne et les chaînes populate, select et sort.vitest

Services testés

ambianceService

Le fichier ambianceService.test.js contient **9 tests**.

Les tests couvrent notamment :

- normalize avec une chaîne normale ;
- normalize avec une valeur absente ;
- normalize avec plusieurs espaces et des underscores ;
- classifyAmbianceRelative avec une valeur basse ;
- classifyAmbianceRelative avec une valeur médiane ;
- classifyAmbianceRelative sans données ;
- calculateAmbianceSummary avec des mesures et observations ;
- le calcul du nombre de mesures ;
- le calcul de la moyenne sonore et le cas où aucune mesure n'est disponible.

Ces tests vérifient principalement les fonctions de calcul et de transformation des données, qui peuvent être exécutées sans serveur.

userService

Le fichier userService.test.js contient **3 tests**.

Les fonctions testées sont :

- formatUser avec des informations publiques ;
- la vérification que le mot de passe et les autres données sensibles ne sont pas retournés ;
- generateToken avec un utilisateur valide.

Le test de formatUser confirme que la réponse contient uniquement :

- text
- id
- name
- email
- role

Le mot de passe, le refresh token et les autres informations sensibles ne sont pas exposés. Le test de generateToken vérifie que la fonction retourne une chaîne de caractères non vide. Les fonctions registerUserService et loginUserService ne sont pas encore testées directement. Elles nécessiteraient des mocks supplémentaires pour User.findOne, User.create, bcrypt.hash et bcrypt.compare.

measurementService

Le fichier measurementService.test.js contient **3 tests**.

Les cas testés sont :

1. refus d'une mesure lorsque soundLevel ou amplitude est absent ;
2. création réussie d'une mesure avec des données valides ;
3. récupération des mesures avec populate et tri par date de création.

Le test de récupération vérifie que le service utilise :

```
.populate("deviceId", "name location")
```

```
.sort({ createdAt: -1 })
```

Le test de création vérifie également que le timestamp par défaut est généré lorsque aucun timestamp n'est fourni.

observationService

Le fichier observationService.test.js contient **4 tests**.

Les cas testés sont :

1. refus d'une observation sans location ou vibe ;
2. création d'une observation associée à un device ;

3. création d'une observation associée à un utilisateur ;
4. récupération des observations avec populate et tri par date.

Pour une observation provenant d'un device, le test vérifie que :

deviceId: "device123"

author: **null**

Pour une observation provenant d'un utilisateur, il vérifie que :

deviceId: **null**

author: "user123"

Le service sépare les données de l'observation des informations d'authentification, ce qui facilite le test de chaque scénario.

deviceService

Le fichier deviceService.test.js contient **3 tests**.

Les cas testés sont :

1. refus d'un device sans name ou location ;
2. création réussie d'un device avec une clé API ;
3. récupération des devices sans exposer les clés API.

Le test de création vérifie que :

- les espaces inutiles sont supprimés ;
- Device.create reçoit les bons paramètres ;
- la clé API est générée ;
- la clé API contient 64 caractères hexadécimaux.

Le test de récupération vérifie l'utilisation de :

Device.find().select("-apiKey");

Cela empêche d'exposer la clé API dans les réponses générales.

locationService

Le fichier locationService.test.js contient **4 tests**.

Les cas testés sont :

1. refus d'un lieu lorsqu'un champ obligatoire est absent ;
2. refus d'un lieu déjà existant ;

3. création réussie d'un lieu ;
4. récupération des lieux triés par nom.

Les tests vérifient notamment que idlocation est normalisé :

"BIBLIO" → "biblio"

Ils vérifient aussi que les espaces sont supprimés avant la création et que les coordonnées égales à 0 peuvent être traitées comme des valeurs valides.

Pour la récupération, le service est vérifié avec :

```
.select("idlocation name latitude longitude")
```

```
.sort({ name: 1 })
```

Résultat global

La suite complète contient actuellement :

Service	Tests
ambianceService	9
userService	3
measurementService	3
observationService	4
deviceService	3
locationService	4
Total	26

Tous les tests passent sans démarrer le serveur et sans utiliser une vraie base de données MongoDB.

Ce qui est testé et ce qui ne l'est pas

Fonctionnalités testées

- validation des champs obligatoires ;
- traitement des données valides ;
- gestion des données absentes ;
- détection des doublons ;

- normalisation des lieux ;
- création des documents ;
- génération des clés API ;
- génération des tokens ;
- protection des données sensibles ;
- tri et récupération des données ;
- association entre observations, utilisateurs et devices ;
- calculs principaux du service d'ambiance.

Fonctionnalités non testées directement

- les routes Express complètes ;
- les contrôleurs avec de vraies requêtes HTTP ;
- l'authentification de bout en bout ;
- la connexion réelle à MongoDB ;
- l'inscription complète avec hashage du mot de passe ;
- la connexion complète avec comparaison du mot de passe ;
- les tests d'intégration entre plusieurs services ;
- le cache backend de la Tâche 4.

Les tests actuels sont volontairement unitaires. Ils isolent les services des modèles MongoDB afin de vérifier la logique métier rapidement et de manière reproductible.

Tâche 4 — Cache backend

1. Objectif

L'objectif de cette tâche est de réduire les lectures répétées dans MongoDB en réutilisant temporairement les réponses des endpoints publics. La stratégie retenue utilise un cache côté serveur, en mémoire, tout en excluant les opérations d'écriture et les réponses contenant des informations privées ou d'authentification.

2. Données pouvant être mises en cache

Seules les réponses publiques obtenues avec la méthode GET sont admissibles au cache :

- GET /locations, pour les informations publiques et relativement stables sur les lieux ;

- GET /devices, uniquement lorsque la réponse ne contient aucune clé API ni donnée privée ;
- GET /measurements, lorsque les mesures retournées sont publiques ;
- GET /observations, lorsque les observations retournées sont publiques ;
- les endpoints publics d'ambiance, comme le résumé d'ambiance, les heures calmes et les prévisions d'ambiance, lorsqu'ils ne contiennent pas de données privées.

Les paramètres de requête font partie de la clé du cache. Ainsi, deux URL différentes, par exemple GET:/measurements et GET:/measurements?location=salle-biblio, correspondent à deux entrées distinctes.

3. Données exclues du cache

Les éléments suivants ne sont jamais mis en cache :

- les requêtes POST, PUT, PATCH et DELETE ;
- les routes de connexion et d'inscription ;
- les réponses contenant un jeton JWT, une clé API, un mot de passe ou des identifiants d'authentification ;
- les réponses contenant des données privées associées à un utilisateur authentifié ;
- toute requête qui contient un en-tête Authorization ou x-api-key.

Le middleware vérifie la méthode HTTP avant toute opération de cache. Il ignore également les requêtes possédant un token ou une clé API :

```
if (req.method !== "GET") {
  return next();
}
```

```
if (req.headers.authorization || req.headers["x-api-key"]) {
  return next();
}
```

Cette décision empêche notamment qu'une réponse privée soit réutilisée pour un autre utilisateur.

4. Durées retenues

Les durées sont adaptées à la fréquence de modification des données :

Type de réponse	Durée retenue	Justification
GET /locations	1 heure	Les lieux changent peu fréquemment.
GET /measurements	60 secondes	Les mesures peuvent changer régulièrement.
GET /observations	60 secondes	Les observations peuvent être ajoutées régulièrement.
Endpoints publics d'ambiance	1 à 5 minutes, selon la route	Les résultats sont recalculés périodiquement.
Devices publics	5 à 15 minutes, selon la route	Les informations changent moins souvent que les mesures.

Dans l'implémentation et les vérifications réalisées, les endpoints principaux utilisent donc les valeurs suivantes :

- GET /locations : 3600 secondes ;
- GET /measurements : 60 secondes ;
- GET /observations : 60 secondes.

Ces valeurs sont envoyées au client avec l'en-tête Cache-Control, par exemple :

Cache-Control: public, max-age=60

5. Implémentation côté serveur

Le cache est conservé dans une structure Map en mémoire. Chaque entrée contient la réponse et son heure d'expiration :

```
const cache = new Map();

function getCacheKey(req) {
  return `${req.method}:${req.originalUrl}`;
}
```

Le middleware construit une clé avec la méthode HTTP et l'URL complète. Il recherche ensuite une réponse existante dans le service de cache. Si l'entrée est encore valide, elle est retournée directement sans nouvelle lecture dans MongoDB.

Lorsqu'il n'y a aucune entrée valide, le middleware intercepte res.json. Après l'exécution du contrôleur, seules les réponses réussies, dont le statut est compris entre 200 et 299, sont enregistrées :

```
if (res.statusCode >= 200 && res.statusCode < 300) {
```



```
setCache(cacheKey, data, ttlMs);  
res.set("X-Cache", "MISS");  
}
```

Les réponses réutilisées contiennent l'en-tête :

X-Cache: HIT

Une première requête non présente dans le cache produit donc X-Cache: MISS. Les requêtes suivantes produisent X-Cache: HIT jusqu'à l'expiration ou l'invalidation de l'entrée.

6. Invalidation après une écriture

Le cache est invalidé lorsqu'une nouvelle mesure ou observation est créée. L'écriture dans MongoDB doit être terminée avant l'invalidation. Ainsi, la prochaine lecture ne peut pas réutiliser une ancienne réponse.

Exemple pour les mesures :

```
deleteCacheByPrefix("GET:/measurements");
```

Exemple pour les observations :

```
deleteCacheByPrefix("GET:/observations");
```

L'utilisation d'un préfixe invalide également les variantes avec paramètres de requête, comme :

```
GET:/measurements?location=salle-a
```

Le même principe est utilisé pour les lieux lorsqu'une nouvelle location est créée :

```
deleteCacheByPrefix("GET:/locations");
```

7. Validation avec Postman et curl

Les opérations de création ont été testées avec Postman. Une requête POST /measurements valide retourne 201 Created. La requête d'écriture peut nécessiter une authentification :

- pour une clé API, ajouter x-api-key dans l'onglet **Headers** ;
- pour un JWT, choisir **Authorization** → **Bearer Token** et coller le token.

Le corps d'une création de mesure testée est de la forme :

```
{  
  "soundLevel": -48,  
  "amplitude": 48  
}
```

Une fois la mesure créée, le cache est vérifié avec :

```
curl.exe -s -D - -o NUL http://localhost:3000/measurements | Select-String "HTTP/[X-  
Cache|Cache-Control"
```

La séquence attendue est :

GET initial → X-Cache: MISS

GET suivant → X-Cache: HIT

POST /measurements → 201 Created

GET suivant → X-Cache: MISS

GET suivant → X-Cache: HIT

La même logique a été vérifiée pour les observations et les locations :

GET /observations : MISS puis HIT, Cache-Control: public, max-age=60

GET /locations : MISS puis HIT, durée de cache d'une heure

La réponse 409 Conflict observée lors d'une inscription avec une adresse déjà utilisée est également un comportement attendu de l'API : elle indique que l'utilisateur existe déjà. Cette réponse d'authentification ou d'inscription ne doit pas être mise en cache.

8. Tests automatisés

Une suite de tests dédiée au service de cache a été ajoutée. Elle couvre notamment :

- l'enregistrement et la récupération d'une valeur ;
- la suppression d'une clé précise ;
- la suppression par préfixe ;
- le comportement d'une clé inexistante ;
- l'expiration d'une entrée.

La commande exécutée est :

```
npm test
```

Résultat obtenu :

```
Test Files 7 passed (7)
```

```
Tests 31 passed (31)
```

9. Limites et choix d'architecture

Le cache utilise une Map en mémoire du processus Node.js. Il est donc simple et adapté au livrable, mais il est vidé lorsque le serveur redémarre et n'est pas partagé entre plusieurs instances de l'API.

Une solution distribuée comme Redis pourrait être envisagée dans un environnement de production comportant plusieurs instances.

Conclusion

La stratégie respecte les exigences de la tâche : seules les réponses publiques de lecture sont mises en cache, les durées sont adaptées au type de données, les écritures ne sont jamais cachées, les tokens et données privées sont exclus, et les entrées concernées sont invalidées après la création de nouvelles mesures, observations ou locations. Les tests automatisés et les vérifications Postman/curl confirment le fonctionnement MISS → HIT ainsi que l'invalidation après écriture.

Tâche 5 – Déploiement

Une partie importante de la vérification finale a été de tester les fonctionnalités directement sur la version déployée et pas seulement en local. Comme le frontend et le backend sont déployés séparément, il fallait s'assurer que le frontend communiquait correctement avec le backend en ligne.

Adresse du backend déployé

Le backend de l'application est accessible à l'adresse suivante :

<https://ift3225-livable1-ambiance-api-1.onrender.com>

Cette adresse a également été ajoutée au fichier README afin de faciliter l'accès à la version déployée de l'API.

Configuration de l'API

Lors des premiers tests sur la version déployée, certaines fonctionnalités qui fonctionnaient en local ne fonctionnaient plus correctement. Certaines requêtes du frontend utilisaient encore localhost au lieu de l'adresse du backend déployé.

Il a donc fallu vérifier la configuration de l'adresse de l'API et les variables d'environnement utilisées lors du déploiement. Après les modifications et un nouveau déploiement, l'inscription et la connexion ont été testées directement sur la version en ligne afin de vérifier que le frontend communiquait correctement avec le backend.

Vérification des fonctionnalités

Plusieurs parcours ont ensuite été testés sur la version déployée, notamment l'inscription, la connexion, l'espace compte et l'accès aux portraits des lieux.

Les tests ont permis de remarquer un problème avec certains portraits. Dans certains cas, l'application utilisait l'identifiant MongoDB _id, alors que les données d'ambiance étaient associées à idlocation. Cela pouvait rendre un portrait indisponible même si des données existaient pour le lieu.

La navigation a donc été corrigée afin d'utiliser le bon identifiant pour accéder au portrait du lieu.

Ces problèmes nous ont montré l'importance de tester l'application après son déploiement. Une fonctionnalité peut fonctionner correctement en local, mais rencontrer des problèmes une fois en ligne à cause de la configuration ou des données utilisées.

Vidéo de démonstration

Une vidéo de démonstration a été réalisée afin de présenter le fonctionnement de l'application et les principales fonctionnalités développées pendant cette phase.

Lien vers la vidéo de démonstration :

https://udemontreal-my.sharepoint.com/:v/g/personal/meriem_ghaoui_umontreal_ca/IQA1Fw5cI1VyT54GultorLt6ARVRdC9-1pduHhccmvpnjL0?e=h0g8M4&nav=eyJyZWZlcnJhbEluZm8iOncicmVmZXJyYWxBcHAIoiJTdHJlYW1XZWJBcHAIiLCJyZWZlcnJhbFZpZXciOiJTaGFyZURpYWxvZy1MaW5rliwicmVmZXJyYWxBcHBQbGF0Zm9ybSI6IldlYiIsInJlZmVycmFsTW9kZSI6InZpZXcifX0%3D

Tâche 6 — Optimisations et faiblesses du backend

6.1 - Backend

1. Objectif

Cette tâche consiste à repérer les traitements répétitifs ou coûteux du backend, à réduire les calculs et lectures inutiles, à mesurer les améliorations observées et à documenter les limites qui restent présentes. Les optimisations réalisées se concentrent principalement sur la réutilisation des réponses publiques avec un cache serveur en mémoire et sur l'invalidation ciblée après les opérations d'écriture.

2. Traitements répétitifs ou coûteux identifiés

Plusieurs traitements peuvent être répétés lorsqu'un même endpoint est appelé plusieurs fois :

- lecture répétée des mêmes lieux dans MongoDB avec GET /locations ;
- lecture répétée des mêmes mesures avec GET /measurements ;
- lecture répétée des mêmes observations avec GET /observations ;
- recalcul ou récupération répétée des résultats des endpoints publics d'ambiance ;
- sérialisation répétée de réponses JSON identiques ;
- vérification et traitement de requêtes identiques avant expiration des données.

Ces répétitions sont particulièrement visibles lorsqu'un client recharge fréquemment une page ou lorsque plusieurs clients consultent les mêmes données publiques sur une courte période. Les opérations d'écriture, comme POST /measurements, ne sont pas considérées comme des opérations à mettre en cache. Elles doivent continuer à écrire dans MongoDB et à invalider les réponses de lecture devenues obsolètes.

3. Optimisation réalisée : cache backend

L'optimisation principale consiste à utiliser un cache serveur en mémoire basé sur une structure Map. La clé est construite à partir de la méthode HTTP et de l'URL complète :

```
const cacheKey = `${req.method}:${req.originalUrl}`;
```

Cette clé différencie également les paramètres de requête. Par exemple, les deux réponses suivantes sont conservées séparément :

GET:/measurements

GET:/measurements?location=salle-a

Le middleware ne traite que les requêtes GET. Lorsqu'une réponse valide existe dans le cache, elle est retournée directement sans nouvelle lecture dans MongoDB :

```
if (req.method !== "GET") {  
  return next();  
}  
  
const cachedResponse = getCache(cacheKey);  
  
if (cachedResponse !== undefined) {  
  res.set("X-Cache", "HIT");  
  return res.status(200).json(cachedResponse);  
}
```

Lorsqu'une réponse n'est pas encore en cache, le contrôleur est exécuté. La réponse réussie est ensuite enregistrée pendant une durée limitée. Le client reçoit alors X-Cache: MISS. Cette optimisation évite de refaire le même accès à MongoDB pour des requêtes publiques identiques durant la période de validité du cache.

4. Durées utilisées

Les durées sont choisies selon la fréquence de modification des données :

Endpoint ou type de données	Durée	Raisonnement
GET /locations	3600 secondes	Les lieux sont relativement stables.
GET /measurements	60 secondes	Les mesures peuvent changer fréquemment.
GET /observations	60 secondes	Les observations peuvent être ajoutées fréquemment.
Endpoints publics d'ambiance	1 à 5 minutes selon la route	Les résultats peuvent être recalculés périodiquement.

Le middleware ajoute également un en-tête HTTP indiquant la durée de validité :

Cache-Control: public, max-age=60

Le cache n'est activé que pour les réponses publiques. Les requêtes qui contiennent Authorization ou x-api-key sont ignorées afin d'éviter la réutilisation d'une réponse privée.

5. Invalidation du cache

Un cache améliore les performances, mais il peut retourner une réponse ancienne si les données changent. Pour éviter ce problème, les entrées concernées sont supprimées après une création réussie.

Après la création d'une mesure :

```
deleteCacheByPrefix("GET:/measurements");
```

Après la création d'une observation :

```
deleteCacheByPrefix("GET:/observations");
```

Après la création d'une location :

```
deleteCacheByPrefix("GET:/locations");
```

La suppression par préfixe permet aussi d'invalider les URLs qui contiennent des paramètres, comme GET:/measurements?location=salle-a.

Après une écriture, la première requête de lecture retourne donc X-Cache: MISS, puis les requêtes suivantes retournent X-Cache: HIT jusqu'à l'expiration ou une nouvelle invalidation.

6. Réduction des calculs inutiles

Le cache réduit principalement les traitements suivants :

- les appels répétitifs aux contrôleurs de lecture ;
- les requêtes identiques envoyées à MongoDB ;

- la conversion répétée des documents récupérés en réponses JSON ;
- l'exécution répétée de calculs pour les réponses publiques d'ambiance, lorsque ces routes sont placées derrière le middleware de cache.

L'optimisation ne supprime pas les calculs nécessaires lors de la première requête. Elle évite plutôt de les répéter pour les requêtes identiques qui surviennent avant l'expiration de l'entrée. Les opérations d'écriture ne sont pas contournées par le cache. Elles continuent à être exécutées normalement afin de garantir que les nouvelles données sont enregistrées.

7. Mesure des améliorations

Les améliorations ont été vérifiées avec les en-têtes de réponse et les tests automatisés.

Vérification par les en-têtes

Pour une route publique comme GET /measurements, la séquence observée est :

Première requête → X-Cache: MISS

Deuxième requête → X-Cache: HIT

Le résultat MISS signifie que le contrôleur et la base de données ont été utilisés avant l'enregistrement de la réponse. Le résultat HIT signifie que la réponse a été réutilisée depuis le cache.

La même séquence a été vérifiée pour les observations et les locations :

GET /measurements → MISS puis HIT

GET /observations → MISS puis HIT

GET /locations → MISS puis HIT

L'en-tête Cache-Control confirme aussi les durées configurées, par exemple :

Cache-Control: public, max-age=60

Vérification après écriture

Après une création réussie avec POST /measurements, la prochaine lecture retourne MISS. Cela confirme que l'ancienne entrée a été supprimée. Une lecture immédiatement répétée retourne ensuite HIT.

La séquence complète est donc :

GET initial → MISS

GET suivant → HIT

POST /measurements → 201 Created

GET suivant → MISS

GET suivant → HIT

Tests automatisés

La suite de tests a été exécutée avec :

npm test

Résultat obtenu :

Test Files 7 passed (7)

Tests 31 passed (31)

Les tests du cache couvrent notamment l'enregistrement, la récupération, la suppression par clé, la suppression par préfixe, l'expiration et les clés inexistantes. Ces résultats démontrent que le comportement fonctionnel du cache est stable. Ils ne constituent toutefois pas un benchmark chiffré du temps de réponse ou du nombre exact de requêtes MongoDB évitées.

8. Limites restantes et pistes d'amélioration

Limite restante	Conséquence	Piste d'amélioration
Le cache utilise une Map locale au processus Node.js.	Les entrées sont perdues lors d'un redémarrage et ne sont pas partagées entre plusieurs instances.	Utiliser Redis ou un autre cache distribué avec une expiration native.
Les réponses sont conservées en mémoire.	Un grand nombre de clés ou de réponses volumineuses peut augmenter l'utilisation de la RAM.	Ajouter une limite de taille, une politique LRU et des métriques de mémoire.
Les durées TTL sont configurées manuellement.	Une durée trop longue peut servir des données anciennes ; une durée trop courte réduit le gain.	Centraliser les TTL dans un fichier de configuration et les ajuster à partir de mesures réelles.
L'invalidation dépend des services d'écriture.	Un chemin d'écriture oublié peut laisser une réponse obsolète dans le cache.	Centraliser l'invalidation ou utiliser une stratégie d'événements après modification des données.
L'invalidation par préfixe peut supprimer plusieurs entrées.	Elle peut provoquer des MISS supplémentaires pour plusieurs variantes de requêtes.	Utiliser des tags de cache ou une indexation plus précise par ressource.

Limite restante	Conséquence	Piste d'amélioration
Les performances n'ont pas encore été mesurées avec un outil de charge.	Le gain réel en millisecondes et en requêtes par seconde n'est pas quantifié.	Utiliser Autocannon, k6 ou Artillery et comparer les résultats avec et sans cache.
Le cache est principalement adapté aux réponses publiques.	Les données privées ne bénéficient pas de cette optimisation.	Mettre en place un cache privé isolé par utilisateur, seulement après une analyse de sécurité.
Les réponses complètes peuvent être stockées.	Une réponse volumineuse peut consommer beaucoup de mémoire et de bande passante.	Ajouter pagination, projection MongoDB et compression lorsque cela est pertinent.
Les requêtes MongoDB doivent encore être optimisées séparément.	Le cache ne règle pas les requêtes lentes lors d'un MISS.	Ajouter des index sur les champs filtrés et analyser les requêtes avec explain().

9. Améliorations qui pourraient être mesurées dans une prochaine étape

Pour obtenir une mesure plus précise, il faudrait comparer au minimum :

- le temps moyen de réponse sans cache et avec cache ;
- le nombre de requêtes MongoDB exécutées pour une série de requêtes identiques ;
- le taux de HIT et de MISS ;
- l'utilisation mémoire du processus Node.js ;
- le comportement sous plusieurs utilisateurs et plusieurs instances.

Un scénario de test pourrait envoyer 100 requêtes identiques à GET /measurements, d'abord sans cache, puis avec cache. Le résultat devrait montrer moins de lectures MongoDB et un temps de réponse inférieur pour les requêtes HIT. Cette mesure n'a pas été prétendue dans le présent livrable, car aucun benchmark de charge chiffré n'a été fourni.

Conclusion

L'optimisation principale réalisée est la mise en cache des réponses publiques de lecture. Elle réduit les lectures et calculs répétés pendant la durée de vie des entrées, tout en maintenant la cohérence grâce à l'invalidation après les écritures. Les en-têtes MISS, HIT et Cache-Control, ainsi que les 31 tests passants, confirment le fonctionnement de la solution.

La principale faiblesse restante est le caractère local et non distribué du cache en mémoire. La piste prioritaire est donc l'utilisation de Redis, accompagnée de métriques de performance et d'un véritable test de charge. D'autres améliorations concernent les index MongoDB, la pagination, la limitation de la mémoire et la centralisation de l'invalidation.

6.2 - Frontend

Limites du système de recommandation

Une des principales limites du système de recommandation est qu'il dépend des données disponibles pour les différents lieux. Si un lieu possède peu de mesures ou si ses dernières données sont anciennes, la recommandation peut être moins représentative de l'ambiance actuelle du lieu.

Il est également possible qu'aucun lieu ne corresponde au besoin choisi par l'utilisateur. Dans ce cas, l'application ne peut pas proposer de recommandation et affiche plutôt un message indiquant qu'aucun résultat n'est disponible.

Le système utilise aussi un nombre limité de catégories d'ambiance. La recommandation reste donc assez simple et ne permet pas encore à l'utilisateur de combiner plusieurs préférences. Une amélioration future pourrait permettre de prendre en compte plusieurs critères, par exemple le type d'ambiance recherché, la proximité du lieu ou la date des dernières données disponibles.

Identification des lieux

Une autre faiblesse observée pendant cette phase concerne l'utilisation de plusieurs identifiants pour représenter les lieux, notamment `_id` et `idlocation`. Cette différence peut créer des problèmes lorsqu'une page utilise un identifiant différent de celui attendu par une autre partie de l'application.

Une amélioration serait donc d'utiliser le même identifiant pour les lieux dans l'ensemble de l'application. Cela permettrait de simplifier la navigation entre les pages et de réduire les risques d'erreurs lors de la récupération des données.